University of Petra

307307
Generative AI

Introduction to Retrieval Augmented Generation (RAG)

The Business Problem and RAG Solution

The Problem: ChatGPT Doesn't Know Your Business

- Imagine you ask ChatGPT:
- "What is our company's refund policy?"
- "Summarize last quarter's financial report"
- "What did the CEO say in yesterday's meeting?"

ChatGPT cannot answer these questions. Why?

- It was trained on public internet data (up to 2023)
- It doesn't have access to your company documents
- It doesn't know your specific business information

More Importantly, ChatGPT might hallucinate and generate fake answers.

Traditional Solutions (and Why They Don't Work)

Option 1: Fine-tune the AI model

1. Cost: $50,000 - $500,000
2. Time: Weeks to months
3. Problem: Needs retraining whenever data changes

Option 2: Just copy-paste documents into ChatGPT

1. Problem: ChatGPT has message limits
2. Problem: Can only handle small documents
3. Problem: Not scalable for hundreds of documents
4. The Solution: RAG (Retrieval Augmented Generation)

RAG is a system that:

1. Stores all your company documents
2. When someone asks a question, finds relevant information
3. Gives that information to ChatGPT
4. ChatGPT answers based on YOUR documents

Real-World Example:

User asks: "What is our remote work policy?"

Without RAG:

- ChatGPT: "I don't have access to your company's policies..."

With RAG:

- System finds your HR policy document
- Extracts the remote work section
- Gives it to ChatGPT
- ChatGPT: "According to your company policy, employees can work remotely up to 3 days per week with manager approval..."

Business Benefits of RAG

1. Cost Effective:

- No expensive model training
- Use existing AI models (ChatGPT, Gemini, etc.)

2. Always Up-to-Date:

- Add new documents anytime
- Answers reflect latest information

3. Accurate:

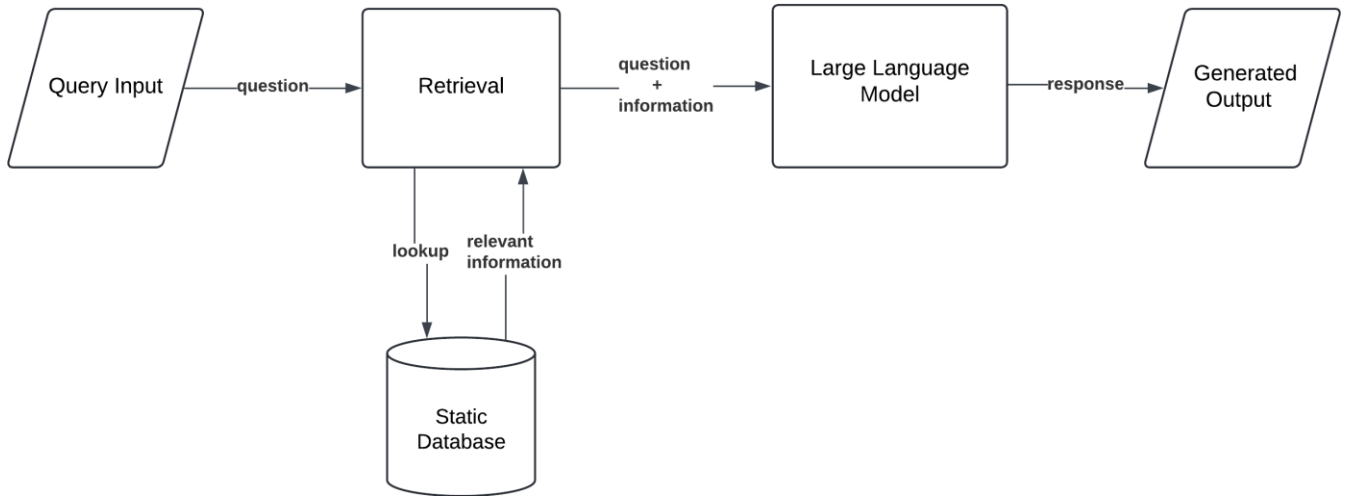
- Answers based on actual company documents
- Less hallucination (making things up)

4. Scalable:

- Works with 10 documents or 10,000 documents

RAG - High-Level Idea

- Retrieve relevant documents from a knowledge source.
- Provide retrieved text to the LLM as context.
- LLM generates a final response using both the prompt and retrieved evidence.



The RAG Process - 3 Main Steps

Step 1: PREPARATION (Do this once)

- Collect all your company documents (PDFs, Word files, etc.)
- Break them into small chunks (like paragraphs)
- Store them in a special database

Step 2: RETRIEVAL (Do this every question)

- User asks a question
- System searches through all chunks
- Finds the 3-5 most relevant chunks
- Like Google search, but for your documents

Step 3: GENERATION (Do this every question)

- Take the question + relevant chunks
- Send to ChatGPT/Gemini
- Get an answer based on your actual documents

Simple Analogy

Think of RAG like a smart librarian:

Traditional ChatGPT:

- You ask a question
- The librarian answers from memory
- Might be wrong or outdated

RAG System:

- You ask a question
- The librarian quickly searches the library
- Finds relevant books
- Reads the relevant pages to you
- Gives you an accurate answer

What Makes RAG Work?

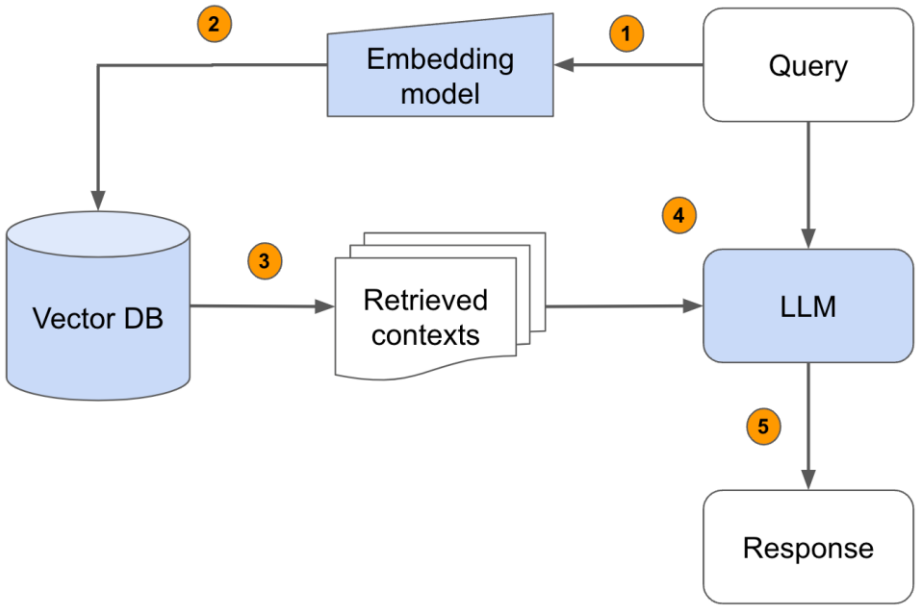
- For RAG to work, we need:
- A way to store documents - **Regular Database? NO**
- A way to search by meaning - **Google Search? NO**
- A way to find similar content - **Need something special!**
- This is where Vector Databases come in.

What RAG Is Not

- It does not modify or fine-tune the LLM weights.
- It does not store new facts inside the model.
- It depends on retrieval quality and good chunking.

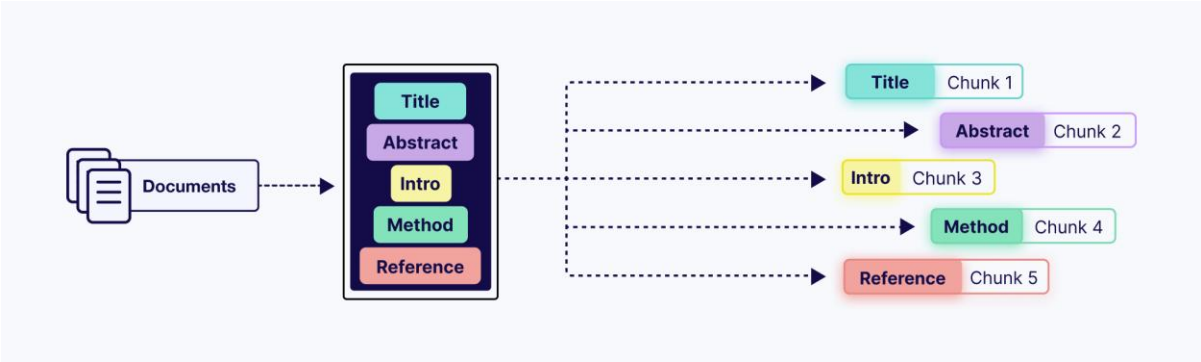
RAG Architecture Components

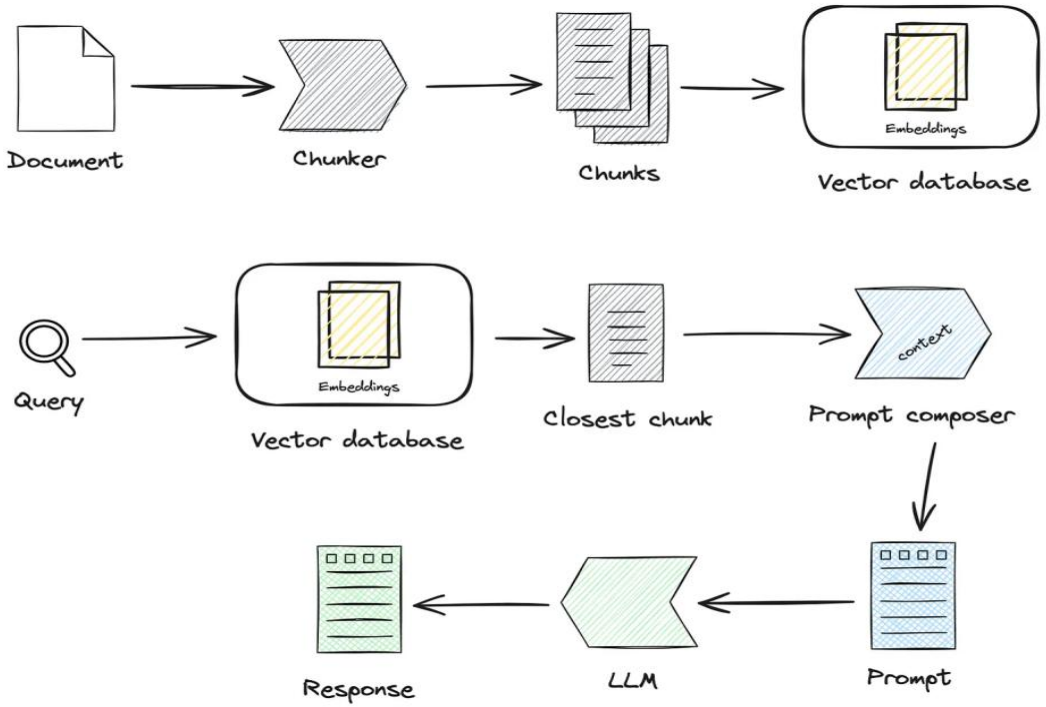
- Embedding model: converts text into vectors.
- Vector store: holds embeddings for fast similarity search.
- Retriever: finds top-k relevant documents.
- LLM: uses retrieved documents to construct an answer.



Document Chunking

- Large documents must be split into smaller chunks.
- Typical sizes: 200–500 tokens.
- Balanced chunks improve retrieval relevance and reduce noise.



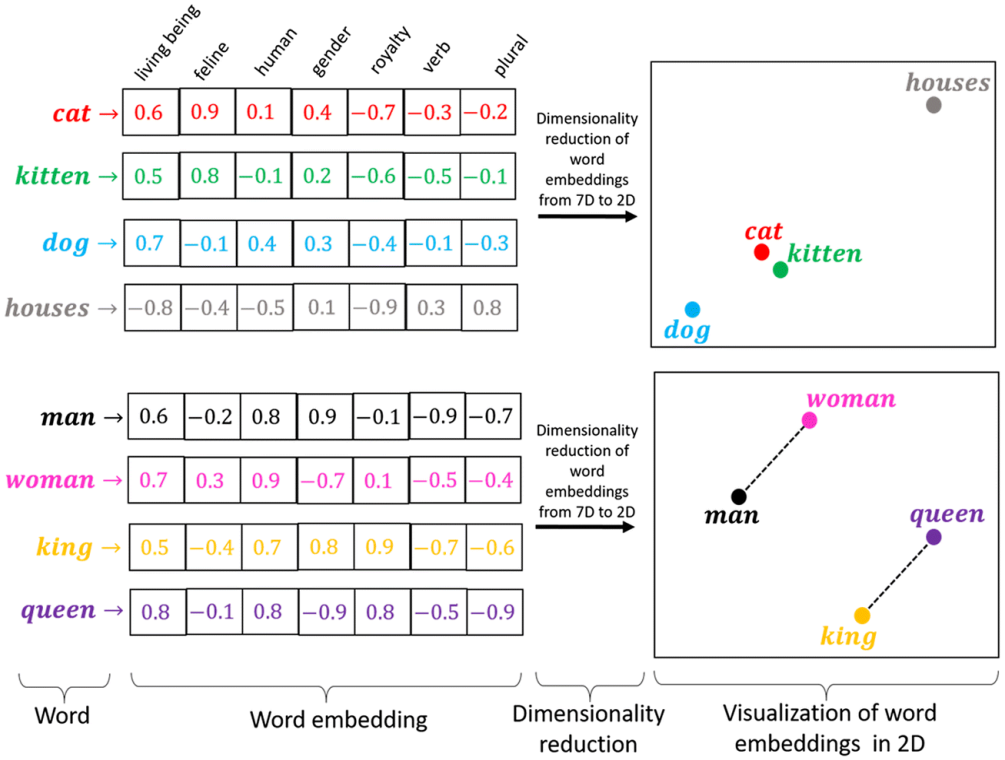


Embeddings

An embedding turns text into a vector in a high-dimensional space.

Similar meaning → vectors close together.

Used for document similarity and semantic search.



Vector Databases - The Missing Piece

Why Regular Databases Don't Work for RAG

Regular Database (SQL):

- Finds exact matches
- "Find customer with email = john@example.com" ✓
- "Find documents about remote work policy" ✗

Google-style Search:

- Finds keyword matches
- "remote work" finds documents with those exact words
- Misses documents saying "telecommuting policy" or "work from home guidelines"

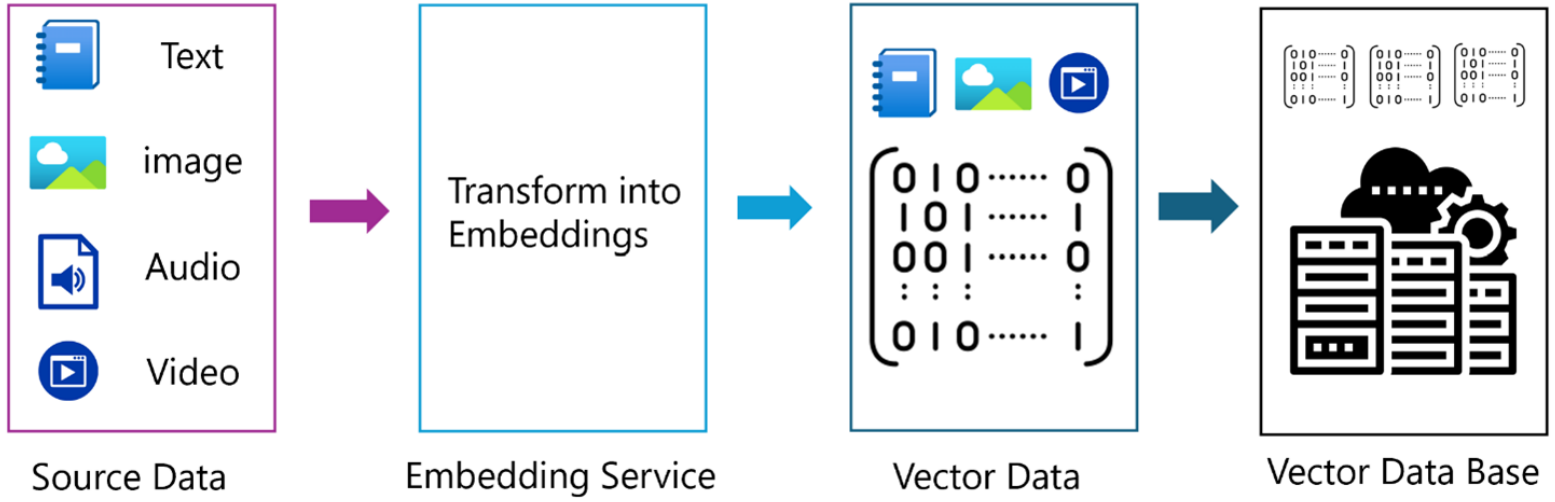
What We Need:

- Search by meaning, not just keywords
- Find similar content even with different words
- This is what vector databases do

Vector Databases

Vector databases are specialized databases that store, index, and search high-dimensional vector embeddings, which are numerical representations (arrays of numbers) of complex, unstructured data like text, images, or audio, enabling AI to find data based on semantic similarity (meaning) rather than just keywords.

- Store text chunks + embeddings.
- Common options: FAISS, Pinecone, Weaviate, Chroma.
- Support nearest-neighbor search at scale.



What is a Vector Database?

Example:

You have 3 documents:

- Doc 1: "Employees can work remotely 3 days per week"
- Doc 2: "Our offices are located in New York"
- Doc 3: "Telecommuting is allowed with manager approval"

Question: "What is the remote work policy?"

Regular database: No exact match found Vector database: Returns Doc 1 and Doc 3 (they're about the same topic)

How Vector Databases Work (Simple View)

Storage:

- Each document chunk gets a vector (list of numbers)
- Database organizes these vectors efficiently
- Like organizing books by topic, not alphabetically

Search:

- Your question becomes a vector
- Database finds vectors close to your question vector
- Returns the original text of those chunks

Key Feature: Similarity Score

- Database tells you how similar each result is
- Score from 0 to 1 (higher = more similar)
- You can filter out low-scoring results

Popular Vector Databases

Chroma:

- Free and open source
- Easy to set up
- Runs on your computer

Pinecone:

- Cloud-based
- Pay per usage
- Good for production systems

Weaviate, Milvus:

- More advanced features
- For larger companies

Putting It All Together

Complete RAG Architecture

Component 1: Document Processor

- Takes your PDFs, Word files
- Extracts text
- Breaks into chunks (500 words each)
- Why chunks? ChatGPT has limits on how much text it can read

Component 2: Embedding Model

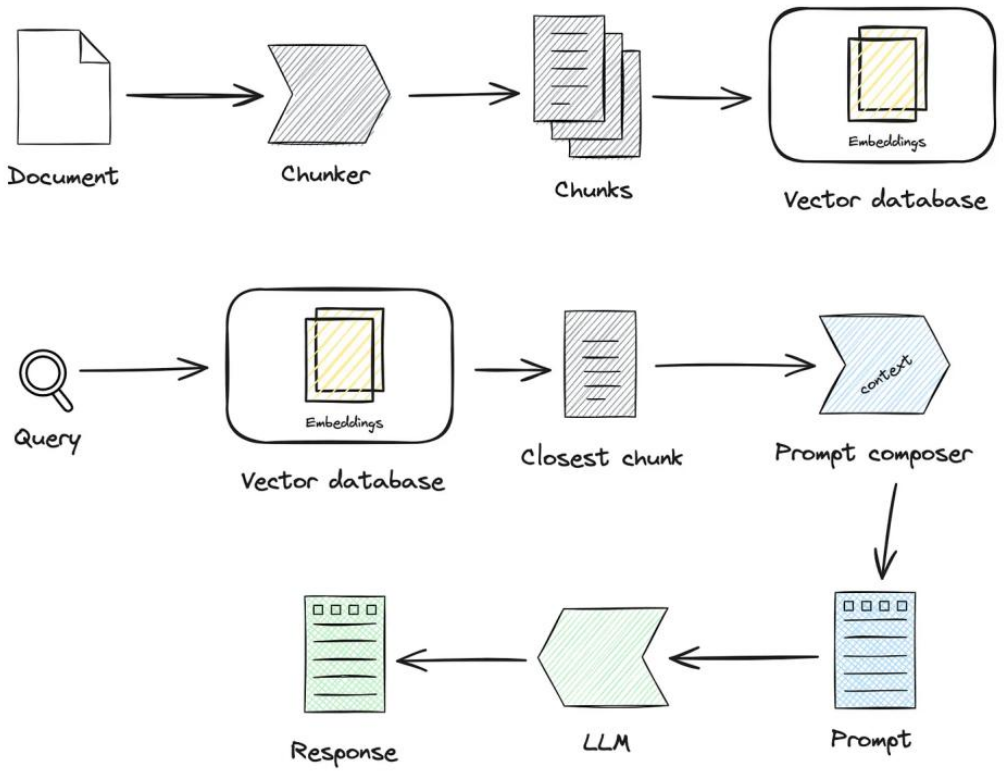
- Converts text chunks to vectors
- Converts user questions to vectors
- You already learned about this

Component 3: Vector Database

- Stores all the vectors
- Performs similarity search
- Returns relevant chunks

Component 4: AI Model (ChatGPT/Gemini)

- Receives question + relevant chunks
- Generates final answer
- Cites sources from your documents



Data Flow Example - Setup Phase

Upload "Employee_Handbook.pdf"

↓

Extract text (50 pages)

↓

Break into 100 chunks

↓

Convert each chunk to vector

↓

Store in Vector Database

Question Answering

User: "How many vacation days do I get?"

↓

Convert question to vector

↓

Search Vector Database

↓

Find top 3 relevant chunks:

- Chunk 47: "Full-time employees receive 15 vacation days..."
- Chunk 48: "Vacation days accrue monthly..."
- Chunk 51: "Unused vacation days can be carried over..."

↓

Send to Gemini: "Here are relevant excerpts from the handbook: [chunk 47, 48, 51]
Question: How many vacation days do I get?"

↓

Gemini: "According to the employee handbook, full-time employees receive 15 vacation days per year..."

Important Concepts

Chunking Strategy:

- Too small chunks: Missing context
- Too large chunks: Irrelevant information mixed in
- Sweet spot: 300-500 words per chunk

Number of Retrieved Chunks:

- Too few: Might miss important information
- Too many: Confuses the AI with too much text
- Common: 3-5 chunks

Cost Considerations:

- Embedding conversion: Usually very cheap
- Vector database storage: Often free for small datasets
- AI generation: This is where most cost is (per question)

When to Use RAG

- You have large or evolving textual knowledge.
- You need factual accuracy and citations.
- You want to update knowledge without retraining a model.
- You want lower cost than fine-tuning.

Simple RAG Example

- Task: Ask questions about a set of product manuals, policies, or PDFs.
- Approach: Chunk → Embed → Retrieve → Generate.

Minimal Python Setup

Below: a compact example using FAISS and a generic LLM API (OpenAI style).
No heavy boilerplate.

```
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np


# Sample documents
docs = [
    "Our warranty covers manufacturing defects for 1 year.",
    "Battery life is approximately 10 hours under normal use.",
    "To reset the device, hold the power button for 5 seconds."
]


# Step 1: Embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")


# Step 2: Embed documents
doc_embeds = model.encode(docs)


# Step 3: Build vector store
index = faiss.IndexFlatL2(doc_embeds.shape[1])
index.add(np.array(doc_embeds))
```

Simple Retrieval Example

```
query = "How long does the battery last?"
query_vec = model.encode([query])


# Retrieve top 1
D, I = index.search(np.array(query_vec), k=1)
retrieved_doc = docs[I[0][0]]


print("Retrieved:", retrieved_doc)
```

Passing Retrieved Context to an LLM

```
import os
from google import genai


# Initialize Gemini client
client = genai.Client(api_key=os.environ["GOOGLE_API_KEY"])


context = retrieved_doc
prompt = f"""
Answer the question using the context below.

Context:
{context}

Question:
{query}
"""

# Call a Gemini chat / text model
response = client.models.generate_content(
    model="gemini-1.5-flash",  # or "gemini-1.5-pro"
    contents=prompt,
)

# Print the text output
print(response.text)
```

Key Observations

- Retrieval controls factual grounding.
- LLM becomes a reasoning and summarization layer.
- Better chunks → better retrieval → better final answer.

Limitations

- If retrieval is poor, the LLM answer will be poor.
- Very long context may exceed model limits.
- Requires well-structured document preprocessing.
- Not ideal for tasks requiring deep parameter-level adaptation.

Improving RAG Quality

- Better chunking strategies (semantic splitting, sentence-based).
- Metadata filtering (section, category, document type).
- Multi-step retrieval (query rewriting, iterative search).
- Re-ranking retrieved chunks using a cross-encoder.

When to Prefer Fine-Tuning Over RAG

Use fine-tuning when:

- You want output style or behavior consistency.
- Task is classification or structured generation.

Use RAG when:

- You need fresh, external knowledge.
- You want answers grounded in documents.

Practice Exercise

Exercise: Design a RAG System for Your Company, Pick one of these scenarios:

Scenario 1: HR Department

- Documents: Employee handbook, policies, benefits guide
- Users: All employees
- Typical questions: "What is the sick leave policy?" "How do I apply for parental leave?"

Scenario 2: Customer Support

- Documents: Product manuals, FAQ, troubleshooting guides
- Users: Support team
- Typical questions: "How to reset password?" "What if product doesn't turn on?"

Scenario 3: Sales Team

- Documents: Product catalogs, pricing sheets, case studies
- Users: Sales representatives
- Typical questions: "What features does Product X have?" "Do we have a case study for manufacturing clients?"

Answer These Questions:

- What documents would you include in your RAG system?
- Who would use this system?
- What are 5 common questions users would ask?
- How would you measure if the system is working well?
- What challenges might you face?

Summary

RAG solves a real business problem:

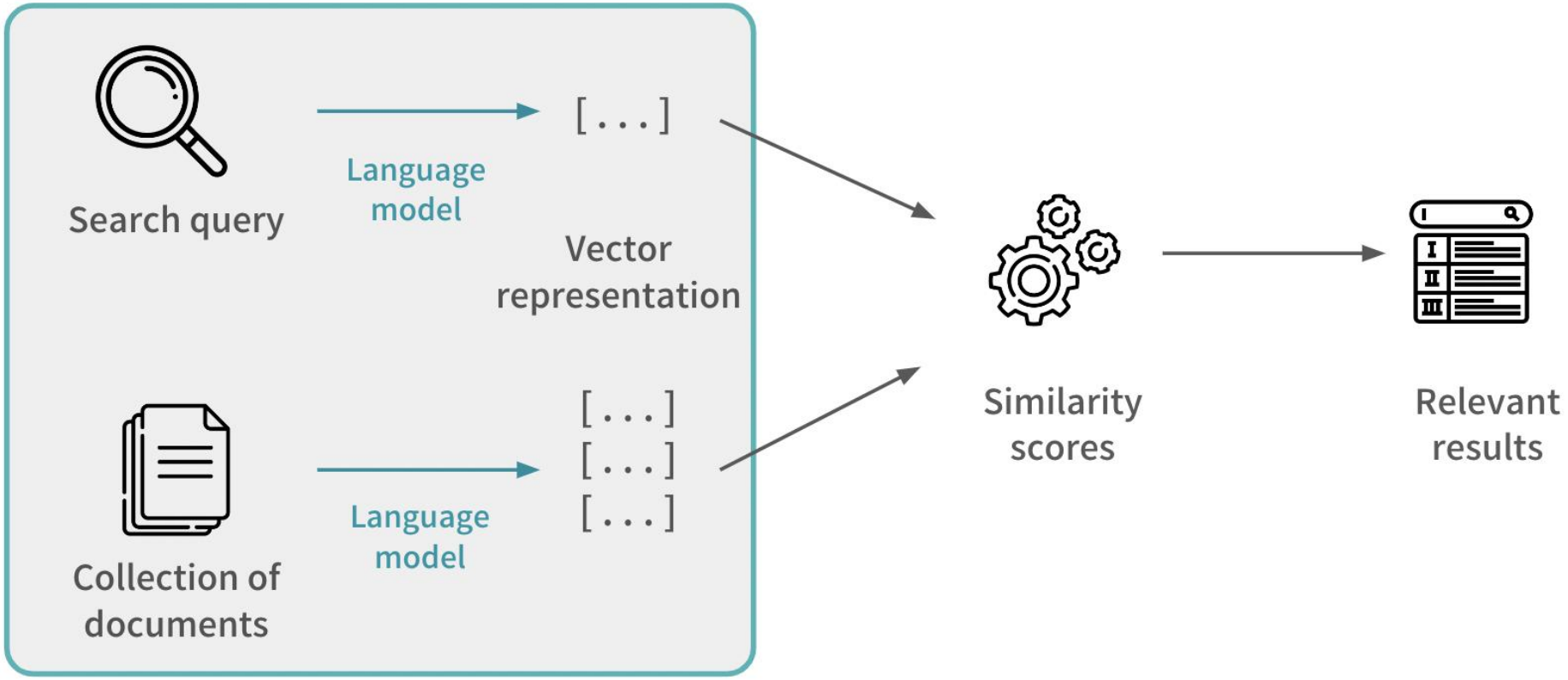
- Enables AI to answer questions about YOUR documents
- More accurate than ChatGPT alone
- Cost-effective compared to custom AI training

Vector Databases are essential for RAG:

- Enable semantic search (search by meaning)
- Store and retrieve document chunks efficiently
- Return similarity scores to ensure relevance

Complete RAG System has 4 components:

- Document processor (chunking)
- Embedding model (text to vectors)
- Vector database (storage and search)
- AI model (answer generation)

Key Takeaway: RAG transforms generic AI assistants into knowledgeable systems that understand your specific business context and documents.

Semantic search